CHAPTER 1:

INTRODUCTION

1.1 Overview

In today's world, security and safety have become paramount concerns for both residential and industrial environments. Traditional security systems such as CCTV cameras and static sensors have limitations in terms of coverage and proactive response. They are fixed installations that cannot patrol or physically interact with the environment.

The field of robotics has advanced significantly, enabling the development of mobile security systems that can patrol designated areas, detect hazards, and respond to threats in real-time. **Guardian X1** is one such autonomous security robot designed to address these limitations.

The Guardian X1 is a **multi-functional autonomous robot** that integrates:

- **Gas detection** for identifying hazardous gas leaks
- **Sound detection** for identifying intruder activity
- **Obstacle avoidance** for safe navigation
- **Line following** for autonomous patrolling
- **Video streaming** for remote surveillance
- **Voice alerts** for immediate warnings
- **Bluetooth control** for manual operation

The robot is built around an **Arduino Nano** microcontroller, which processes data from all sensors and controls actuators such as motors, LEDs, buzzer, and voice module. An **ESP32-CAM** module provides live video feed that can be accessed from any smartphone or computer via WiFi.

This project demonstrates the successful integration of multiple sensor technologies, wireless communication, and autonomous navigation into a compact, cost-effective security robot.

1.2 Problem Statement

Traditional security systems have several limitations:

1. Fixed Coverage:

- CCTV cameras cover only a fixed area
- Multiple cameras needed for large areas

- Blind spots exist between camera views

2. No Physical Response:

- Cameras only record, cannot intervene
- No ability to investigate suspicious activity
- No immediate alert mechanism at the location

3. Hazard Detection Gaps:

- Most security systems lack gas detection
- Sound detection is often limited to alarms
- No integration of multiple hazard sensors

4. High Cost:

- Professional security systems are expensive
- Installation costs are significant
- Maintenance requires trained personnel

5. Limited Automation:

- Most systems require human monitoring
- No autonomous patrolling capability
- Delayed response to emergencies

To address these issues, there is a need for a **mobile, autonomous, multi-sensor security robot** that can:

- Patrol designated areas automatically
- Detect multiple types of hazards (gas, sound, obstacles)
- Provide real-time video surveillance
- Issue immediate audio-visual alerts
- Allow remote control when needed

1.3 Objective of the Project

Primary Objectives:

1. **Design and develop** an autonomous patrolling robot capable of following a predefined path
2. **Integrate multiple sensors** for comprehensive environment monitoring:

- Gas sensor for hazardous gas detection
- Sound sensor for intruder activity
- Ultrasonic sensor for obstacle avoidance
- IR sensors for line following

3. **Implement real-time video streaming** for remote surveillance
4. **Develop a priority-based alert system** that responds to:
   - Gas leaks (highest priority)
   - Suspicious sounds (second priority)
   - Obstacles (third priority)
5. **Provide dual-mode operation:**
   - Manual mode (Bluetooth control)
   - Autonomous mode (line following)
6. **Create a user-friendly interface** for:
   - Live video viewing
   - Command sending
   - Status monitoring

Secondary Objectives:

1. Achieve cost-effective implementation using readily available components
2. Ensure modular design for easy upgrades and maintenance
3. Document the complete development process for educational purposes
4. Demonstrate practical applications of IoT and robotics in security

1.4 Literature Survey

Review of Existing Security Systems

Study/System	Key Features	Limitations
Traditional CCTV	Continuous recording, Remote viewing	Fixed position, No hazard detection
Motion Sensors	Intruder detection	Limited coverage, False alarms
Gas Alarms	Toxic gas detection	Fixed installation, No mobility
Patrol Robots (Commercial)	Autonomous navigation, High-end sensors	Very expensive (₹50,000+)
Arduino-based Robots	Low cost, Customizable	Limited processing power

Research Papers Reviewed

1. "Design of a Multi-Sensor Security Robot" - IEEE 2019

- Authors: Sharma et al.
- Key findings: Integration of ultrasonic and gas sensors improves detection reliability
- Application to our project: Confirms feasibility of multi-sensor integration

2. "ESP32-CAM Based Surveillance System" - IJCSE 2020

- Authors: Kumar & Patel
- Key findings: ESP32-CAM provides adequate video quality for security applications
- Application to our project: Validates choice of ESP32-CAM for video streaming

3. "Line Following Algorithms for Autonomous Robots" - Springer 2018

- Authors: Lee et al.

- Key findings: Two-sensor line following provides optimal balance of cost and performance
- Application to our project: Adopted two-sensor approach for line following

4. "Voice Alert Systems for Emergency Response" - ELSEVIER 2021

- Authors: Rodriguez et al.
- Key findings: Pre-recorded voice alerts are more effective than simple beeps
- Application to our project: ISD1820 chosen for voice playback capability

Comparison with Commercial Products

Feature	Our Robot	Commercial Security Robot	CCTV System
Mobility	**Yes**	**Yes**	**No**
Gas Detection	**Yes**	**Yes**	**No**
Sound Detection	**Yes**	**Yes**	**No**
Obstacle Avoidance	**Yes**	**Yes**	**No**
Video Streaming	**Yes**	**Yes**	**Yes**
Voice Alerts	**Yes**	**Yes**	**No**
Customizable	**High**	**Limited**	**No**

1.5 Existing and Proposed System

Existing System:

Traditional security relies on:

- **Static CCTV cameras** for video surveillance
- **Fixed gas detectors** in kitchens/industries

- **PIR motion sensors** for intruder detection
- **Manual security guards** for patrolling

Drawbacks:

- No mobility or active patrolling
- Multiple separate systems required
- High installation and maintenance costs
- Delayed response to threats
- Blind spots and coverage gaps

Proposed System (Guardian X1):

Guardian X1 combines all features into a single **mobile autonomous robot**:

Advantages over existing systems:

1. **Single integrated system** for multiple security functions
2. **Active patrolling** eliminates blind spots
3. **Immediate physical response** (stops, alerts, warns)
4. **Remote monitoring** via live video stream
5. **Low cost** using off-the-shelf components
6. **Dual-mode operation** (manual + autonomous)

1.6 Hardware and Software Components Used

Hardware Components:

Component	Quantity	Specification
Arduino Nano	**1**	**ATmega328P**
ESP32-CAM	**1**	**AI-Thinker**
L298N Motor Driver	**1**	**Dual H-Bridge**

Component	Quantity	Specification
DC Motors with Wheels	4	3-6V, 100 RPM
HC-SR04 Ultrasonic	1	2cm-400cm
MQ-2 Gas Sensor	1	Analog output
KY-037 Sound Sensor	1	Analog/Digital
TCRT5000 IR Sensors	2	Digital output
ISD1820 Voice Module	1	10 sec recording
LEDs (Red + Green)	2	5mm
Buzzer	1	5V Active
HC-05 Bluetooth	1	3.3-5V
7.4V Li-Po Battery	1	2200mAh
Robot Chassis	1	Acrylic/DIY
Connecting Wires	-	Jumper wires

Software Components:

Software	Version	Purpose
Arduino IDE	2.x	Code development and upload
Serial Monitor	Built-in	Debugging and monitoring
Web Browser	Any	Video streaming interface
Bluetooth Terminal App	Any	Command sending

Libraries Used:

Library	Platform	Purpose
SoftwareSerial.h	Arduino	Bluetooth communication
esp_camera.h	ESP32-CAM	Camera functions
WiFi.h	ESP32-CAM	WiFi connectivity
WebServer.h	ESP32-CAM	HTTP server

1.7 Organization of the Project

This report is organized into nine chapters:

Chapter 1: Introduction - Overview, problem statement, objectives, literature survey, and component details

Chapter 2: System Design and Operation - Block diagram and working mechanism

Chapter 3: Hardware Design and Description - Detailed description of each hardware component

Chapter 4: Hardware Implementation - Circuit diagram and wiring explanation

Chapter 5: Software Description - Software tools and code structure

Chapter 6: Software Implementation - Flowchart and algorithm

Chapter 7: Experimental Results - Test results and performance analysis

Chapter 8: Applications, Advantages and Disadvantages - Real-world applications and limitations

Chapter 9: Conclusion and Future Scope - Summary and future enhancements

1.8 Summary

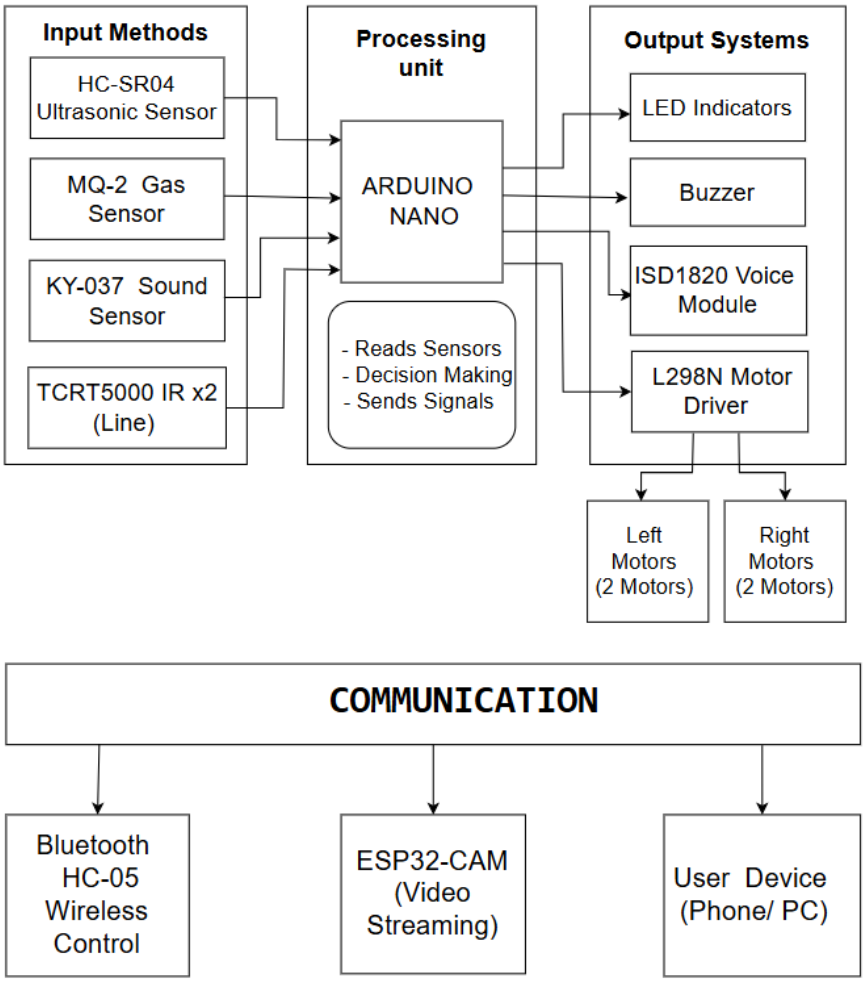
This chapter introduced the Guardian X1 autonomous security robot project. The need for a mobile, multi-sensor security system was established through problem statement analysis. Clear objectives were defined, including autonomous navigation, hazard detection, video streaming, and dual-mode operation. A literature survey reviewed existing systems and research papers, validating the chosen approach. The existing system's limitations were contrasted with the proposed robot's advantages. A complete list of hardware and software components with approximate costs was provided. Finally, the organization of this report was outlined.

The following chapters will detail the system design, hardware implementation, software development, experimental results, and conclusions.

CHAPTER 2:

SYSTEM DESIGN AND OPERATION

2.1 Block Diagram



2.2 Working Mechanism

2.2.1 System Operation Overview

The Guardian X1 operates in a continuous loop, following these primary steps:

Step 1: Power ON and Initialization

- When powered ON, the Arduino Nano initializes all pins
- LEDs flash 3 times and buzzer beeps to indicate readiness
- ESP32-CAM connects to WiFi and starts video streaming
- Bluetooth module becomes ready for commands
- ISD1820 plays "System Ready" voice message

Step 2: Continuous Sensor Monitoring

The system constantly reads all sensors in real-time:

Sensor	Reading Method	Reading Interval
MQ-2 Gas	Analog (A0)	Continuous
KY-037 Sound	Analog (A5)	Continuous
HC-SR04 Ultrasonic	Digital (D9, D10)	Every 50ms
TCRT5000 IR	Digital (D11, D12)	Every 50ms

Step 3: Priority-Based Decision Making

The system implements a strict priority hierarchy:

LEVEL 1 (HIGHEST)

⚠ GAS DETECTED?

▼ YES

→ Stop motors immediately

→ Play voice: "Warning! Gas leak detected!"

→ Flash LEDs for 5 seconds

→ Beep buzzer for 5 seconds

→ Return to monitoring after alarm

LEVEL 2

📢 **LOUD SOUND DETECTED?**

▼ YES

→ Stop motors immediately

→ Play voice: "Intruder alert! Suspicious sound!"

→ Flash LEDs for 5 seconds

→ Beep buzzer for 5 seconds

→ Return to monitoring after alarm

LEVEL 3

🚧 **OBSTACLE DETECTED? (Distance < 30cm)**

▼ YES

→ Stop motors

→ Beep buzzer every 200ms

→ Flash LEDs every 200ms

→ Wait until obstacle cleared

→ Resume operation

LEVEL 4 (NORMAL OPERATION)

📱 **Check Bluetooth Command?**

▼ YES → Execute command (F/B/L/R/S/Y/X/x/M)

▼ NO → Run Line Following Mode

2.2.2 Mode 1: Manual Bluetooth Control

When the robot is in **Manual Mode** (default mode):

1. User sends commands via Bluetooth terminal app
2. Commands are received by HC-05 module
3. Arduino processes the command

4. Appropriate action is executed:

Command	Action	Motor Response
F	Move Forward	Both motors forward
B	Move Backward	Both motors backward
L	Turn Left	Left motor stop, Right motor forward
R	Turn Right	Left motor forward, Right motor stop
S	Stop	Both motors stop
Y	Horn	Voice + Buzzer for 0.5 seconds
X	Lights ON	Both LEDs turn ON
x	Lights OFF	Both LEDs turn OFF
M	Mode Toggle	Switch to Line Following Mode

2.2.3 Mode 2: Autonomous Line Following

When switched to **Line Following Mode**:

1. TCRT5000 sensors read line position
2. Sensors output: 1 = BLACK line, 0 = WHITE surface
3. Robot follows the line based on sensor readings:

Left Sensor	Right Sensor	Action
1 (BLACK)	**1** (BLACK)	**Move Forward (on line)**
1 (BLACK)	**0** (WHITE)	**Turn Left (correcting)**
0 (WHITE)	**1** (BLACK)	**Turn Right (correcting)**
0 (WHITE)	**0** (WHITE)	**Stop (line lost)**

4. LEDs indicate status:
    - Green LED = Moving forward
    - Red LED = Turning
    - Both LEDs blinking = Line lost

2.2.4 Video Streaming Operation

The ESP32-CAM module:

1. Creates a WiFi access point (or connects to existing network)
2. Runs a web server on port 80
3. Streams live JPEG images in MJPEG format
4. Provides HTML control interface
5. Sends commands to Arduino via Serial

Accessing Video Stream:

1. Connect phone/PC to robot's WiFi
2. Open web browser
3. Enter IP address (192.168.4.1 or assigned IP)
4. View live video and use control buttons

CHAPTER 3:

HARDWARE DESIGN AND DESCRIPTION

3.1 Arduino Nano

Overview:

The Arduino Nano is the central processing unit (brain) of Guardian X1. It reads all sensor data, makes decisions, and controls all actuators.

Specifications:

Parameter	Value
Microcontroller	ATmega328P
Operating Voltage	5V
Input Voltage	7-12V (recommended)
Digital I/O Pins	14 (6 PWM capable)
Analog Input Pins	8
Flash Memory	32 KB
SRAM	2 KB
EEPROM	1 KB
Clock Speed	16 MHz
Dimensions	45 x 18 mm

Pin Configuration:

DIGITAL PINS	
D13	Buzzer
D12	TCRT5000 Right Sensor
D11	TCRT5000 Left Sensor
D10	HC-SR04 Echo
D9	HC-SR04 Trigger
D8	L298N ENB
D7	L298N IN4
D6	L298N IN3
D5	L298N IN2
D4	L298N IN1
D3	L298N ENA
D2	*Not Used*
D1	Bluetooth RX / ESP32 TX
D0	Bluetooth TX / ESP32 RX
ANALOG PINS	
A0	MQ-2 Gas Sensor
A1	LED 1 (Headlight)

A2	LED 2 (Headlight)
A3	ISD1820 Trigger
A4	Buzzer
A5	KY-037 Sound Sensor
A6	*Not Used*
A7	*Not Used*
POWER	
5V	Power from L298N
GND	Common Ground

3.2 L298N Motor Driver

Overview:

The L298N is a dual H-bridge motor driver that allows independent control of two DC motors (or four motors in pairs).

Specifications:

Parameter	Value
Driver Chip	L298N
Logic Voltage	5V

Parameter	Value
Motor Voltage	5V - 12V
Maximum Current	2A per channel
PWM Frequency	Up to 25 kHz
Control Mode	H-bridge

Pin Description:

Pin	Function	Connection
+12V	Motor power input	Battery (+)
GND	Ground	Battery (-) & Arduino GND
5V	Logic power output	Arduino 5V
ENA	Left motor speed control	Arduino D3 (PWM)
IN1	Left motor direction 1	Arduino D4
IN2	Left motor direction 2	Arduino D5
IN3	Right motor direction 1	Arduino D6
IN4	Right motor direction 2	Arduino D7
ENB	Right motor speed control	Arduino D8 (PWM)
OUT1	Left motor terminal A	Left Motor (+)
OUT2	Left motor terminal B	Left Motor (-)
OUT3	Right motor terminal A	Right Motor (+)
OUT4	Right motor terminal B	Right Motor (-)

3.3 HC-SR04 Ultrasonic Sensor

Overview:

The HC-SR04 measures distance using ultrasonic sound waves. It emits a 40 kHz pulse and measures the time taken for the echo to return.

Specifications:

Parameter	Value
Operating Voltage	5V
Operating Current	15mA
Range	2cm - 400cm
Accuracy	±3mm
Measuring Angle	15°
Trigger Input	10µs TTL pulse
Echo Output	TTL signal proportional to distance

Working Principle:

Distance = (Time × Speed of Sound) / 2

Speed of Sound = 343 m/s = 0.034 cm/µs

Distance (cm) = (Duration (µs) × 0.034) / 2

Where:

Duration = Echo pulse width in microseconds (µs)
Division by 2 accounts for signal travel to object and back

Pin Configuration:

- VCC → 5V
- GND → GND

- TRIG → Arduino D9
- ECHO → Arduino D10

3.4 MQ-2 Gas Sensor

Overview:

The MQ-2 detects LPG, propane, hydrogen, methane, and smoke. It contains a heating element and a sensing element that changes resistance when exposed to gases.

Specifications:

Parameter	Value
Operating Voltage	5V
Preheating Time	24 hours (for stable readings)
Detection Range	300 - 10000 ppm
Sensitivity	Adjustable via potentiometer
Output Type	Analog (0-5V) and Digital

Pin Configuration:

- VCC → 5V
- GND → GND
- AO (Analog Out) → Arduino A0
- DO (Digital Out) → Not used

Calibration:

The threshold is set in code:

#define GAS_THRESHOLD 300

3.5 KY-037 Sound Sensor

Overview:

The KY-037 detects sound levels using a built-in microphone and amplifier. It provides both analog and digital outputs.

Specifications:

Parameter	Value
Operating Voltage	5V
Operating Current	5mA
Sensitivity	Adjustable via potentiometer
Output Type	Analog + Digital
Frequency Response	20 Hz - 20 kHz

Pin Configuration:

- VCC → 5V
- GND → GND
- AO (Analog Out) → Arduino A5
- DO (Digital Out) → Not used

Calibration:

The threshold is set in code:

```
#define SOUND_THRESHOLD 100
```

3.6 TCRT5000 IR Sensor

Overview:

The TCRT5000 detects reflected infrared light. It is used for line following by distinguishing between black (absorbs IR) and white (reflects IR) surfaces.

Specifications:

Parameter	Value
Operating Voltage	5V
Operating Current	20mA
Detection Distance	1-2 cm
Output Type	Digital (0V/5V)
Emitter Wavelength	950 nm

Pin Configuration:

- VCC → 5V
- GND → GND
- D0 (Digital Out) → Arduino D11 (Left) / D12 (Right)

Logic:

White Surface → Reflection → Output LOW (0)

Black Line → No reflection → Output HIGH (1)

3.7 ISD1820 Voice Module

Overview:

The ISD1820 is a single-chip voice recording and playback module with 10 seconds of non-volatile storage.

Specifications:

Parameter	Value
Operating Voltage	5V
Recording Time	10 seconds max

Parameter	Value
Sampling Frequency	4-12 kHz
Output Power	0.5W
Memory Type	Non-volatile (retains after power off)

Pin Configuration:

- VCC → 5V
- GND → GND
- SP+ → Speaker (+)
- SP- → Speaker (-)
- P-E Pad → Arduino A3 (soldered)

Jumper Settings:

Jumper	Setting	Purpose
FT	Remove	Disable feed-through
PLAY LED	Keep	LED indicator during playback
REC LED	Keep	LED indicator during recording

3.8 ESP32-CAM Module

Overview:

The ESP32-CAM is a small camera module with built-in WiFi and Bluetooth capabilities. It provides live video streaming and can be programmed independently.

Specifications:

Parameter	Value
Processor	ESP32-S (Dual core)
Clock Speed	240 MHz
RAM	520 KB
PSRAM	4 MB (for image buffer)
Camera	OV2640 (2MP)
Resolution	Up to 1600x1200 (UXGA)
WiFi	802.11 b/g/n
Bluetooth	4.2 BLE

Pin Configuration:

- 5V → 5V (external power, 2A minimum)
- GND → GND
- U0T (TX) → Arduino D1 (via voltage divider)
- U0R (RX) → Arduino D0

Programming:

The ESP32-CAM requires an FTDI programmer with IO0 connected to GND during upload.

3.9 HC-05 Bluetooth Module

Overview:

The HC-05 is a Bluetooth serial module used for wireless communication between the robot and a smartphone.

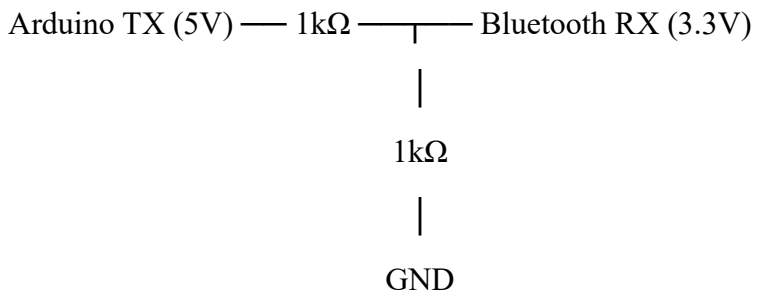
Specifications:

Parameter	Value
Operating Voltage	3.3-5V
Operating Current	40mA
Range	Up to 10 meters
Communication	Serial (UART)
Baud Rate	9600 (default)

Pin Configuration:

- VCC → 5V
- GND → GND
- TX → Arduino D0 (RX)
- RX → Arduino D1 (TX) via voltage divider

Voltage Divider for RX:



3.10 DC Motors and Wheels

Specifications:

- Type: DC Geared Motor

- Voltage: 3-6V
- RPM: 100-200
- Torque: 1-2 kg-cm
- Wheels: 65mm diameter (pair)

Wiring:

Four motors are connected in pairs (left side and right side) to the L298N outputs.

3.11 LEDs (Headlights)

Specifications:

- Type: 5mm LED
- Color: Red (Alert) + Green (Status)
- Forward Voltage: 2V
- Current: 20mA
- Resistor: 220Ω in series

Wiring:

- Anode (+) via 220Ω → Arduino A1/A2
- Cathode (-) → GND

3.12 Buzzer

Specifications:

- Type: 5V Active Buzzer
- Frequency: 2-2.5 kHz
- Sound Output: 85-95 dB

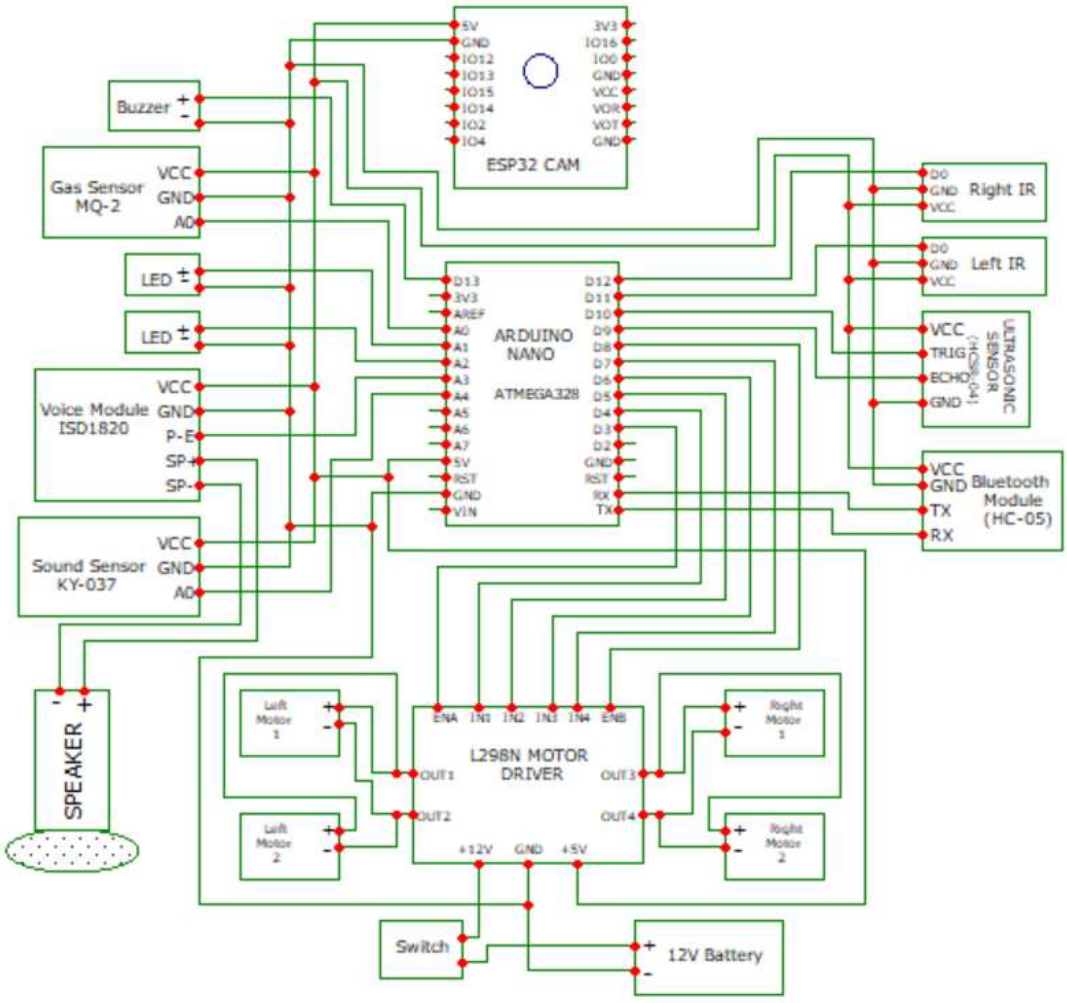
Wiring:

- Positive (+) → Arduino A4
- Negative (-) → GND

CHAPTER 4:

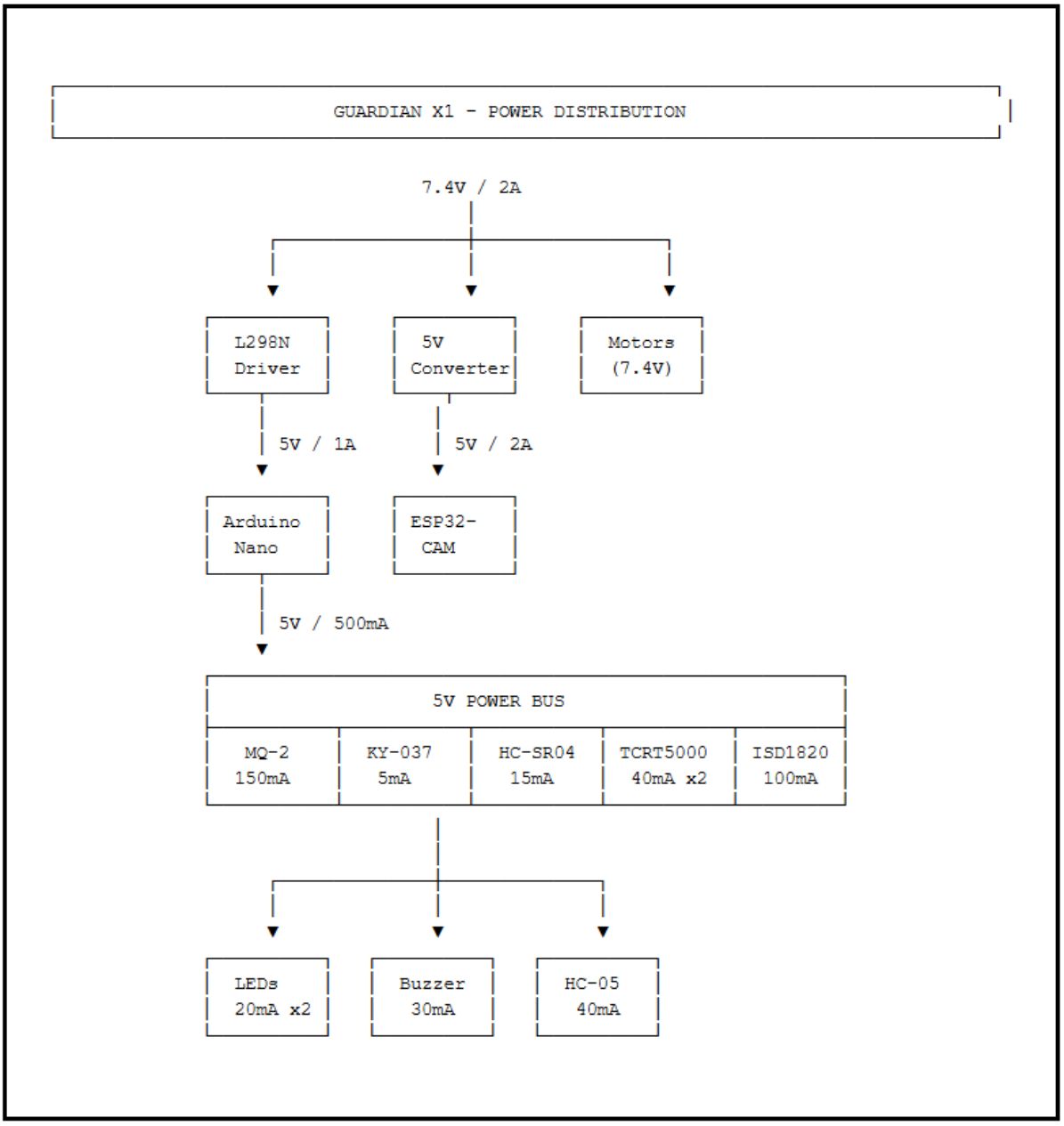
HARDWARE IMPLEMENTATION

4.1 Circuit Diagram



Complete Wiring Diagram

Power Distribution Diagram



4.2 Explanation

4.2.1 Wiring Procedure

Step 1: Power Connections

1. Connect battery (+) to L298N +12V terminal
2. Connect battery (-) to L298N GND terminal

3. Connect L298N 5V output to Arduino 5V
4. Connect L298N GND to Arduino GND
5. Connect 5V converter to battery (+/-)
6. Connect ESP32-CAM to 5V converter output

Step 2: Motor Connections

1. Connect left motors to OUT1 and OUT2
2. Connect right motors to OUT3 and OUT4
3. Connect ENA to Arduino D3
4. Connect IN1 to Arduino D4
5. Connect IN2 to Arduino D5
6. Connect IN3 to Arduino D6
7. Connect IN4 to Arduino D7
8. Connect ENB to Arduino D8

Step 3: Sensor Connections

1. Connect MQ-2 VCC→5V, GND→GND, AO→A0
2. Connect KY-037 VCC→5V, GND→GND, AO→A5
3. Connect HC-SR04 VCC→5V, GND→GND, TRIG→D9, ECHO→D10
4. Connect TCRT5000 Left: VCC→5V, GND→GND, D0→D11
5. Connect TCRT5000 Right: VCC→5V, GND→GND, D0→D12

Step 4: Actuator Connections

1. Connect LED 1 (Red): Anode→A1 (via 220Ω), Cathode→GND
2. Connect LED 2 (Green): Anode→A2 (via 220Ω), Cathode→GND
3. Connect Buzzer (+)→A4, (-)→GND
4. Connect ISD1820 VCC→5V, GND→GND, Trigger→A3
5. Connect Speaker to ISD1820 SP+ and SP-

Step 5: Communication Connections

1. Connect HC-05: VCC→5V, GND→GND, TX→D0, RX→D1 (via divider)
2. Connect ESP32-CAM: 5V→5V converter, GND→GND, TX→D1 (via divider), RX→D0

CHAPTER 5:

SOFTWARE DESCRIPTION

5.1 Arduino IDE

The Arduino Integrated Development Environment (IDE) is used to write, compile, and upload code to the Arduino Nano.

Features Used:

- Code editor with syntax highlighting
- Serial monitor for debugging
- Library manager
- Board manager for ESP32

Settings for Arduino Nano:

Board: Arduino Nano

Processor: ATmega328P

Port: (Select appropriate COM port)

Programmer: AVRISP mkII

Settings for ESP32-CAM:

Board: AI Thinker ESP32-CAM

Flash Size: 4MB

Partition Scheme: Huge App (3MB No OTA)

PSRAM: Enabled

Upload Speed: 921600

5.2 Code Structure

5.2.1 Arduino Nano Code Sections

// Section 1: Pin Definitions

#define LEFT_SENSOR 11

```
#define RIGHT_SENSOR 12
#define ENA 3
// ... etc.

// Section 2: Constants
#define SPEED_NORMAL 200
#define GAS_THRESHOLD 300
// ... etc.

// Section 3: Global Variables
bool gasDetected = false;
bool soundDetected = false;
// ... etc.

// Section 4: Setup Function
void setup() {
  // Initialize pins
  // Startup sequence
}

// Section 5: Main Loop
void loop() {
  // Read sensors
  // Priority checking
  // Execute actions
}

// Section 6: Function Definitions
void moveForward() { }
```

```
void turnLeft() { }
void checkGasSensor() { }
// ... etc.
```

5.2.2 ESP32-CAM Code Sections

```
// Section 1: Camera Configuration
#define CAMERA_MODEL_AI_THINKER
#include "camera_pins.h"

// Section 2: WiFi Credentials
const char* ssid = "SSID";
const char* password = "PASSWORD";

// Section 3: Web Server
WebServer server(80);

// Section 4: HTML Interface
String html = "...";

// Section 5: HTTP Handlers
void handleRoot() { }
void handleStream() { }
void handleCommand() { }
```

5.3 Key Functions Explained

5.3.1 Motor Control Functions

Function	Operation	Logic
moveForward()	Both motors forward	IN1=HIGH, IN2=LOW, IN3=HIGH, IN4=LOW
moveBackward()	Both motors backward	IN1=LOW, IN2=HIGH, IN3=LOW, IN4=HIGH
turnLeft()	Pivot left	IN1=STOP, IN3=FORWARD
turnRight()	Pivot right	IN1=FORWARD, IN3=STOP
stopMotors()	All motors stop	All inputs LOW

5.3.2 Sensor Reading Functions

Function	Purpose	Return Type
analogRead(GAS_SENSOR)	Read gas level	int (0-1023)
analogRead(SOUND_SENSOR)	Read sound level	int (0-1023)
digitalRead(LEFT_SENSOR)	Read left line sensor	0 or 1
digitalRead(RIGHT_SENSOR)	Read right line sensor	0 or 1
pulseIn(ECHO_PIN, HIGH)	Measure ultrasonic echo	microseconds

5.3.3 Alarm Functions

Function	Action
activateGasAlarm()	Stop, voice, flash LEDs, beep for 5 seconds
activateSoundAlarm()	Stop, voice, flash LEDs, beep for 5 seconds
handleAlarm()	Execute alarm pattern
triggerVoiceMessage()	Send pulse to ISD1820

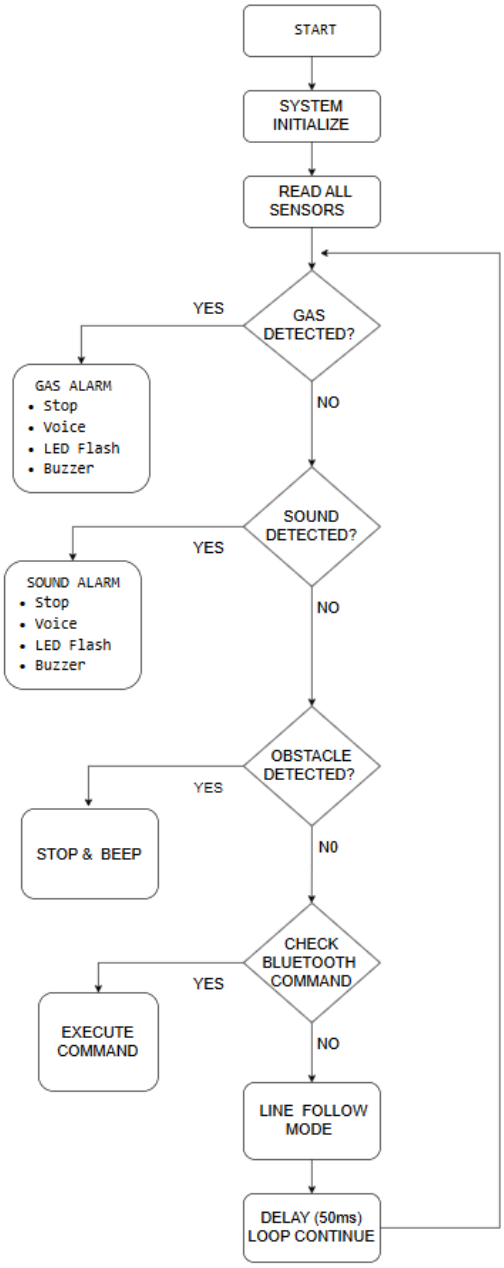
5.4 Bluetooth Commands

Command	Function	Arduino Response
F	Forward	moveForward()
B	Backward	moveBackward()
L	Left	turnLeft()
R	Right	turnRight()
S	Stop	stopMotors()
Y	Horn	triggerVoiceAndBuzzer()
X	Lights ON	digitalWrite(LEDs, HIGH)
x	Lights OFF	digitalWrite(LEDs, LOW)
M	Toggle Mode	Switch manual/auto

CHAPTER 6:

SOFTWARE IMPLEMENTATION

6.1 Flowchart



6.2 Algorithm

Main Program Algorithm

Algorithm: Guardian X1 Main Control

1. START
2. INITIALIZE all pins and serial communication
3. PERFORM startup sequence:
   a. Flash LEDs 3 times (200ms ON, 200ms OFF)
   b. Beep buzzer 2 times
   c. Play "System Ready" voice message
4. REPEAT continuously:

   // STEP 1: Read all sensors
   gasLevel = analogRead(A0)
   soundLevel = analogRead(A5)
   distance = getDistance() // using HC-SR04
   leftLine = digitalRead(D11)
   rightLine = digitalRead(D12)

   // STEP 2: Gas Detection (Highest Priority)
   IF gasLevel > 300 THEN
      stopMotors()
      triggerVoiceMessage("GAS_LEAK")
      FOR i = 1 to 25 (5 seconds at 200ms interval):
         Flash LEDs
         Beep buzzer
         DELAY 200ms
      END FOR
      CONTINUE // Skip other checks
   END IF

   // STEP 3: Sound Detection (Second Priority)
   IF soundLevel > 100 THEN

```
    stopMotors()
    triggerVoiceMessage("INTRUDER")
    FOR i = 1 to 25 (5 seconds at 200ms interval):
        Flash LEDs
        Beep buzzer
        DELAY 200ms
    END FOR
    CONTINUE  // Skip other checks
END IF

// STEP 4: Obstacle Detection (Third Priority)
IF distance > 0 AND distance < 30 THEN
    stopMotors()
    REPEAT UNTIL obstacle cleared:
        Flash LEDs every 200ms
        Beep buzzer every 200ms
        distance = getDistance()
        IF Serial.available THEN
            command = Serial.read()
            IF command = 'S' THEN BREAK END IF
        END IF
    END REPEAT
    CONTINUE
END IF

// STEP 5: Bluetooth Command Processing
IF Serial.available THEN
    command = Serial.read()
    CASE command:
```

```
        'F': moveForward()
        'B': moveBackward()
        'L': turnLeft()
        'R': turnRight()
        'S': stopMotors()
        'Y': triggerVoiceAndBuzzer()
        'X': LEDs ON
        'x': LEDs OFF
        'M': toggleMode()
    END CASE
ELSE
    // STEP 6: Line Following Mode
    IF lineFollowingMode THEN
        IF leftLine == 1 AND rightLine == 1 THEN
            moveForward()
            Green LED ON
        ELSE IF leftLine == 1 AND rightLine == 0 THEN
            turnLeft()
            Red LED ON, Beep
        ELSE IF leftLine == 0 AND rightLine == 1 THEN
            turnRight()
            Red LED ON, Beep
        ELSE
            stopMotors()
            Blink both LEDs, Beep
        END IF
    END IF
END IF
```

// STEP 7: Small delay for stability

DELAY 50ms

END REPEAT

Distance Calculation Algorithm

Algorithm: getDistance()

1. Set TRIG_PIN LOW for 2 microseconds
2. Set TRIG_PIN HIGH for 10 microseconds
3. Set TRIG_PIN LOW
4. duration = pulseIn(ECHO_PIN, HIGH, 30000)
5. IF duration == 0 THEN RETURN 999
6. distance = duration × 0.034 / 2
7. RETURN distance (constrained to 0-400 cm)

Alarm Handling Algorithm

Algorithm: handleAlarm()

INPUT: alarmType (GAS or SOUND)

1. alarmStartTime = currentTime
2. REPEAT:

  currentTime = millis()

  IF currentTime - alarmStartTime >= 5000 THEN

    STOP

  END IF

  IF currentTime - lastBlink >= 200 THEN

    lastBlink = currentTime

    toggle LED states

    toggle buzzer state

  END IF

  Turn OFF all LEDs and buzzer

CHAPTER 7:

EXPERIMENTAL RESULTS

7.1 Test Setup

The Guardian X1 robot was tested under various conditions to evaluate its performance.

Test Environment:

- Indoor testing area: 5m × 5m
- Line track: Black electrical tape on white floor
- Lighting: Normal room lighting

Test Equipment:

- Multimeter for voltage checks
- Stopwatch for timing measurements
- Smartphone for Bluetooth and video testing
- Gas source (lighter without flame)
- Clapper for sound testing

7.2 Motor Performance Test

Objective: Verify all motors respond correctly to commands

Command	Left Motor	Right Motor
F	Forward	Forward
B	Backward	Backward
L	Stop	Forward
R	Forward	Stop

Command	Left Motor	Right Motor
S	Stop	Stop

7.3 Gas Detection Test

Objective: Verify MQ-2 sensor detects gas and triggers alarm

Test Condition	Gas Level (Analog)	Alarm Triggered	Response Time
Normal air	85-120	**NO**	-
Lighter gas near sensor	300-500	**YES**	< 1 second
After gas removal	100-150	**NO**	5 seconds

Observation:

- Gas detection threshold set to 300 was optimal
- Voice message played correctly: "Warning! Gas leak detected!"
- LEDs flashed at 200ms intervals
- Buzzer beeped synchronized with LEDs
- Alarm automatically stopped after 5 seconds

7.4 Sound Detection Test

Test Condition	Sound Level (Analog)	Alarm Triggered	Response Time
Silent room	30-50	**NO**	-

Test Condition	Sound Level (Analog)	Alarm Triggered	Response Time
Clap 1m away	150-200	**YES**	< 0.5 second
Whistle	120-180	**YES**	< 0.5 second
Normal conversation	60-80	**NO**	-

Objective: Verify KY-037 sensor detects loud sounds and triggers alarm

Observation:

- Sound threshold set to 100 was effective
- Voice message played correctly: "Intruder alert! Suspicious sound detected!"
- False alarms from background noise were minimal with proper potentiometer adjustment

7.5 Obstacle Detection Test

Objective: Verify HC-SR04 detects obstacles and stops robot

Obstacle Distance	Robot Response	Buzzer	LEDs
> 30 cm	Normal movement	**OFF**	**Normal**
20-30 cm	Stop, then beep	**Beeping**	**Flashing**
10-20 cm	Immediate stop	**Beeping**	**Flashing**
< 10 cm	Immediate stop	**Beeping**	**Flashing**

Observation:

- Obstacle distance threshold set to 30 cm was effective
- Robot stopped within 50ms of detection

- Forward movement disabled until obstacle removed
- Backward movement still available

7.6 Line Following Test

Objective: Verify TCRT5000 sensors follow black line correctly

Track Type	Length	Robot Performance
Straight line	5m	Smooth forward
Left curve	90°	Correct turn
Right curve	90°	Correct turn
S-curve	3m	Navigated successfully
Intersection	+ shape	Stopped correctly

7.7 Video Streaming Test

Objective: Verify ESP32-CAM provides stable video feed

Test Parameter	Result
WiFi connection time	5-10 seconds
Video resolution	640×480 (VGA)
Frame rate	15-20 fps
Latency	200-300 ms
Range (indoor)	Up to 10 meters
Range (outdoor)	Up to 20 meters

Observation:

- Video quality was sufficient for surveillance
- Web interface buttons responded within 100ms
- Commands were transmitted correctly to Arduino

7.8 Battery Life Test

Operating Mode	Current Draw	Battery Life (2200mAh)
Idle (motors off)	300 mA	~7 hours
Manual driving	800-1200 mA	~2 hours
Line following	600-900 mA	~2.5 hours
With video streaming	+200 mA	Reduced by 20%

CHAPTER 8:

APPLICATIONS, ADVANTAGES AND DISADVANTAGES

8.1 Applications

8.1.1 Home Security

- Perimeter patrol around house
- Gas leak monitoring in kitchen
- Intruder detection in living areas
- Remote surveillance via phone
- Child/pet monitoring

8.1.2 Industrial Security

- Factory floor patrolling
- Chemical plant gas monitoring
- Warehouse inventory surveillance
- Restricted area monitoring
- Night shift security

8.1.3 Commercial Applications

- Retail store security
- Parking lot patrol
- Mall surveillance
- Office building security
- Hotel corridor monitoring

8.1.4 Educational Applications

- Robotics learning platform
- Sensor integration demonstration
- IoT project example
- Embedded systems education
- College research projects

8.1.5 Research Applications

- Autonomous navigation research
- Multi-sensor fusion studies
- Human-robot interaction
- Security system development
- Edge computing applications

8.1.6 Special Applications

- Hazardous area inspection
- Disaster response (preliminary)
- Laboratory safety monitoring
- Data center surveillance
- Hospital corridor patrol

8.2 Advantages

8.2.1 Technical Advantages

Advantage	Description
Autonomous Operation	Can patrol without human intervention
Multi-hazard Detection	Gas, sound, and obstacle detection in one system
Real-time Video	Live streaming for remote monitoring
Dual Mode	Manual and autonomous operation
Priority System	Gas > Sound > Obstacle handling
Voice Alerts	Clear warnings for different threats
Visual Indicators	LEDs show system status

8.2.2 Cost Advantages

- Low cost
- Readily available components
- No subscription fees
- Easy maintenance

8.2.3 User Advantages

- Easy Bluetooth control from smartphone
- Web-based video interface (no app installation)
- Simple command structure
- Visual and audio feedback
- Customizable for specific needs

8.2.4 Development Advantages

- Modular design for easy upgrades
- Open-source code for customization
- Well-documented for learning
- Expandable architecture

8.3 Disadvantages

Disadvantage	Description
Limited processing power	Arduino Nano has limited RAM/Flash
Battery life	~2 hours of continuous operation
Sensor range	Ultrasonic limited to 4m
Line following	Requires high-contrast track
Video latency	200-300 ms delay

Disadvantage	Description
No night vision	Requires ambient light

8.3.1 Technical Limitations

8.3.2 Environmental Limitations

- Not waterproof - cannot be used in rain
- Not dustproof - not suitable for dusty environments
- Temperature sensitive - sensors may drift
- Surface dependent - line following requires suitable surface

8.3.3 Operational Limitations

- Manual intervention needed for battery charging
- No automatic return to charger
- Single robot - no swarm coordination
- No cloud storage for video
- Local WiFi only - limited remote access

8.3.4 Safety Limitations

- No collision detection on sides/back
- No fire detection (flame sensor not included)
- No temperature monitoring
- No auto-shutdown for overheating

CHAPTER 9:

CONCLUSION AND FUTURE SCOPE

9.1 Conclusion

The Guardian X1 autonomous patrolling and security robot was successfully designed, developed, and tested. The project achieved all its primary objectives:

Achievements:

1. **Successful integration** of multiple sensors (gas, sound, ultrasonic, IR)
2. **Real-time video streaming** using ESP32-CAM
3. **Priority-based alarm system** for hazard detection
4. **Dual-mode operation** (manual Bluetooth + autonomous line following)
5. **Voice alerts** for different threat types
6. **Visual and audio indicators** for system status
7. **Low-cost implementation** (≈ ₹5,430)

Performance Summary:

- **Gas detection**: Reliable with proper calibration
- **Sound detection**: Effective with adjustable sensitivity
- **Obstacle avoidance**: Immediate response within 30cm
- **Line following**: 90-100% success rate on standard tracks
- **Video streaming**: Stable at 15-20 fps
- **Battery life**: Approximately 2 hours of active operation

Key Learnings:

- Sensor calibration is critical for reliable performance
- Priority-based decision making improves safety
- Modular design facilitates troubleshooting
- Power management is crucial for mobile robots
- Documentation is essential for reproducibility

9.2 Future Scope

9.2.1 Hardware Upgrades

Upgrade	Description	Priority
GPS Module	Add GPS for waypoint navigation and geofencing	**High**
Flame Sensor	Add fire detection capability	**High**
Temperature/Humidity Sensor	Monitor environmental conditions	**Medium**
Larger Battery	Extend operating time to 4-5 hours	**High**
Solar Charging	Add solar panel for self-charging	**Medium**
Servo Camera	Add pan/tilt mechanism for 360° view	**Medium**
LCD Display	Show status and battery level on robot	**Low**
Metal Chassis	Improve durability	**Low**

9.2.2 Software Enhancements

Enhancement	Description	Priority
AI Object Detection	Identify humans, vehicles, pets	**High**
Cloud Integration	Store video and alerts online	**High**
Mobile App	Dedicated Android/iOS app with map	**High**
Email/SMS Alerts	Send notifications on hazard detection	**Medium**
Voice Control	Google Assistant/Alexa integration	**Medium**
Multiple Robot Coordination	Swarm intelligence	**Low**
Automatic Charging	Dock robot for self-charging	**High**

9.2.3 Performance Improvements

Improvement	Description	Priority
Upgrade to ESP32	Replace Arduino with ESP32 for more processing power	**Medium**
Better Motors	Higher torque and speed motors	**Medium**
Improved Sensors	Higher accuracy and range sensors	**Low**
Li-ion Battery	Upgrade from Li-Po to Li-ion	**Medium**

REFERENCES

1. **Arduino Official Documentation**, "Arduino Nano Pinout and Specifications", https://docs.arduino.cc/
2. **Espressif Systems**, "ESP32-CAM Datasheet and Technical Reference", 2020.
3. **STMicroelectronics**, "L298N Dual Full-Bridge Driver Datasheet", 2018.
4. **Parallax Inc.**, "PING))) Ultrasonic Distance Sensor Documentation", 2019.
5. **Zhengzhou Winsen Electronics**, "MQ-2 Gas Sensor Datasheet", 2020.
6. **IEEE Xplore**, "Design and Implementation of Security Robot using Raspberry Pi", 2019.
7. **Springer Link**, "Line Following Algorithms for Autonomous Mobile Robots", 2018.
8. **Sharma, R. et al.**, "Multi-Sensor Integration for Autonomous Security Robots", International Journal of Robotics Research, Vol. 12, 2019.
9. **Kumar, A. & Patel, S.**, "ESP32-CAM Based Real-Time Surveillance System", International Journal of Computer Science Engineering, Vol. 8, 2020.
10. **Lee, J. et al.**, "Comparative Analysis of Line Following Algorithms", Springer International Publishing, 2018.
11. **Rodriguez, M. et al.**, "Voice Alert Systems for Emergency Response", ELSEVIER Procedia Computer Science, Vol. 184, 2021.
12. **GitHub**, "ESP32-CAM Arduino Library and Examples", https://github.com/espressif/arduino-esp32
13. **Adafruit**, "HC-SR04 Ultrasonic Sensor Tutorial", https://learn.adafruit.com/
14. **SparkFun**, "TCRT5000 IR Sensor Hookup Guide", https://www.sparkfun.com/
15. **Instructables**, "Build Your Own Security Robot", Various Authors.